Assignment 6 jQuery

Team: BookingApp Group: SE-2513

Members: Allazhar Aldongarov, Dias Serikov, Yerkenov Abylay

Objective: Add jQuery search, animations and user feedback to our existing hotel booking website.

Website: https://aishhko.vercel.app

Repository: https://github.com/jjestar/web

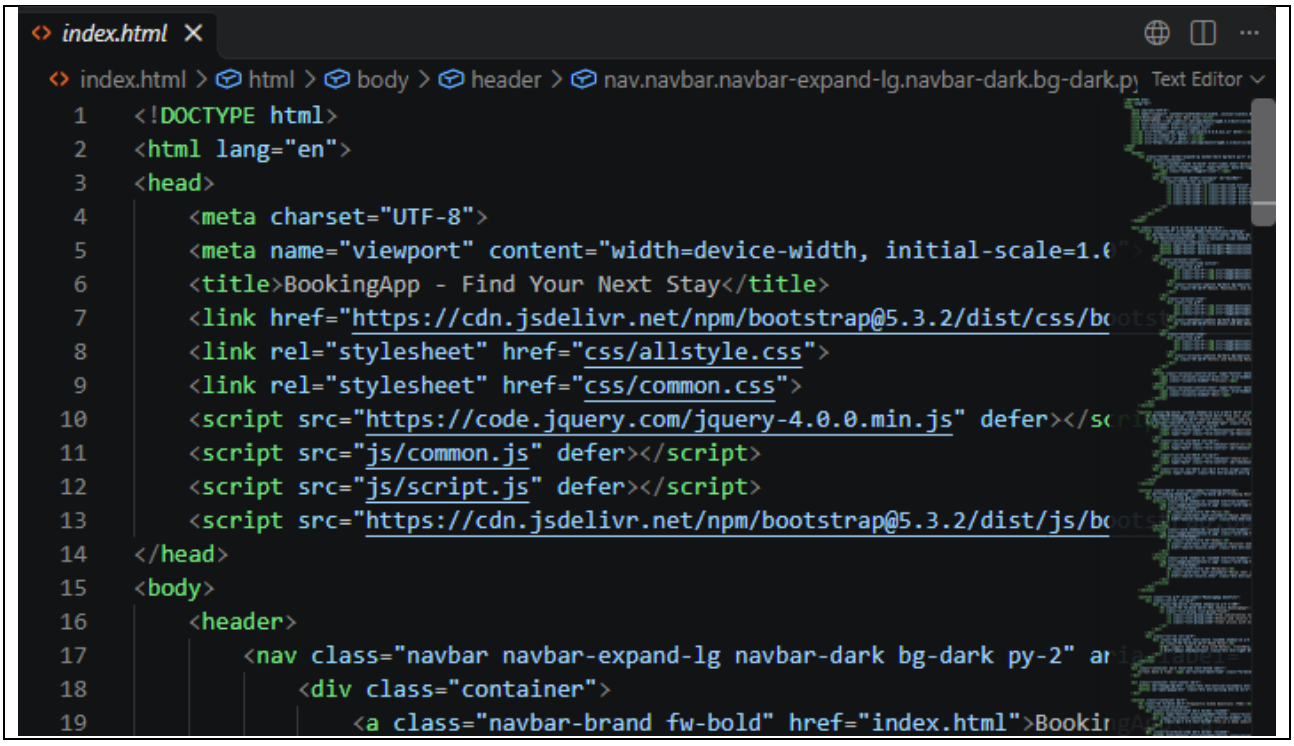
For the code screenshots, keep the VS Code file tab and line numbers visible. Insert each screenshot into the matching empty box.

Task 0 jQuery setup

1. We added jQuery 4.0.0 in the head of each HTML page before our JavaScript files.
2. We used $(document).ready() to run the code after the page is ready.
3. The browser console shows "jQuery is ready!".

VS Code: index.html, lines 3-14

Show the head section and the jQuery CDN script.



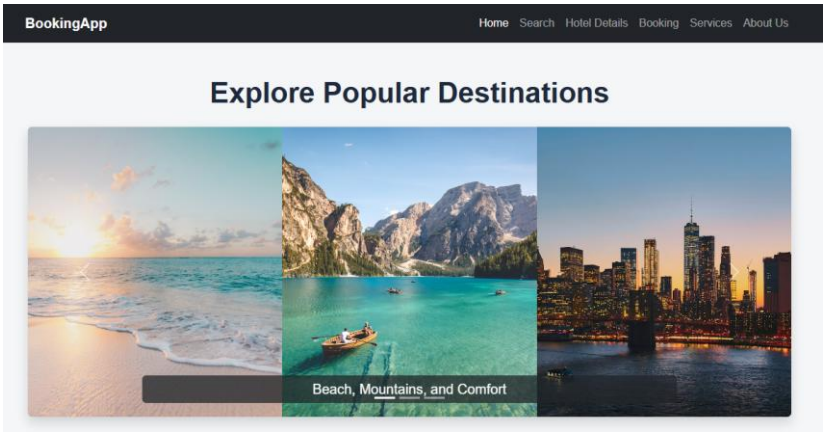
VS Code: js/script.js, lines 1-2

Show $(document).ready() and console.log().

```
JS script.js
js > JS script.js
1   $(document).ready(function () {
2       console.log('jQuery is ready!');
3       const modal = document.getElementById('popup-modal');
4       const openButton = document.getElementById('open-popup-btn');
5       const closeButton = document.getElementById('close-popup-btn');
6       const emailInput = document.getElementById('popup-email');
7
8       function closePopup() {
9           modal.hidden = true;
10          document.body.style.overflow = '';
11          openButton.focus();
12      }
13
14      if (modal && openButton && closeButton) {
15          openButton.addEventListener('click', function () {
16              modal.hidden = false;
17              document.body.style.overflow = 'hidden';
18              emailInput.focus();
19          });
20          closeButton.addEventListener('click', closePopup);
21          modal.addEventListener('click', function (event) {
22              if (event.target === modal) {
23                  closePopup();
24              }
25          });
26          document.addEventListener('keydown', function (event) {
27              if (modal.hidden) {
28                  return;
29              }
30              if (event.key === 'Escape') {
```

Do you want to install extension from tomok

Website result



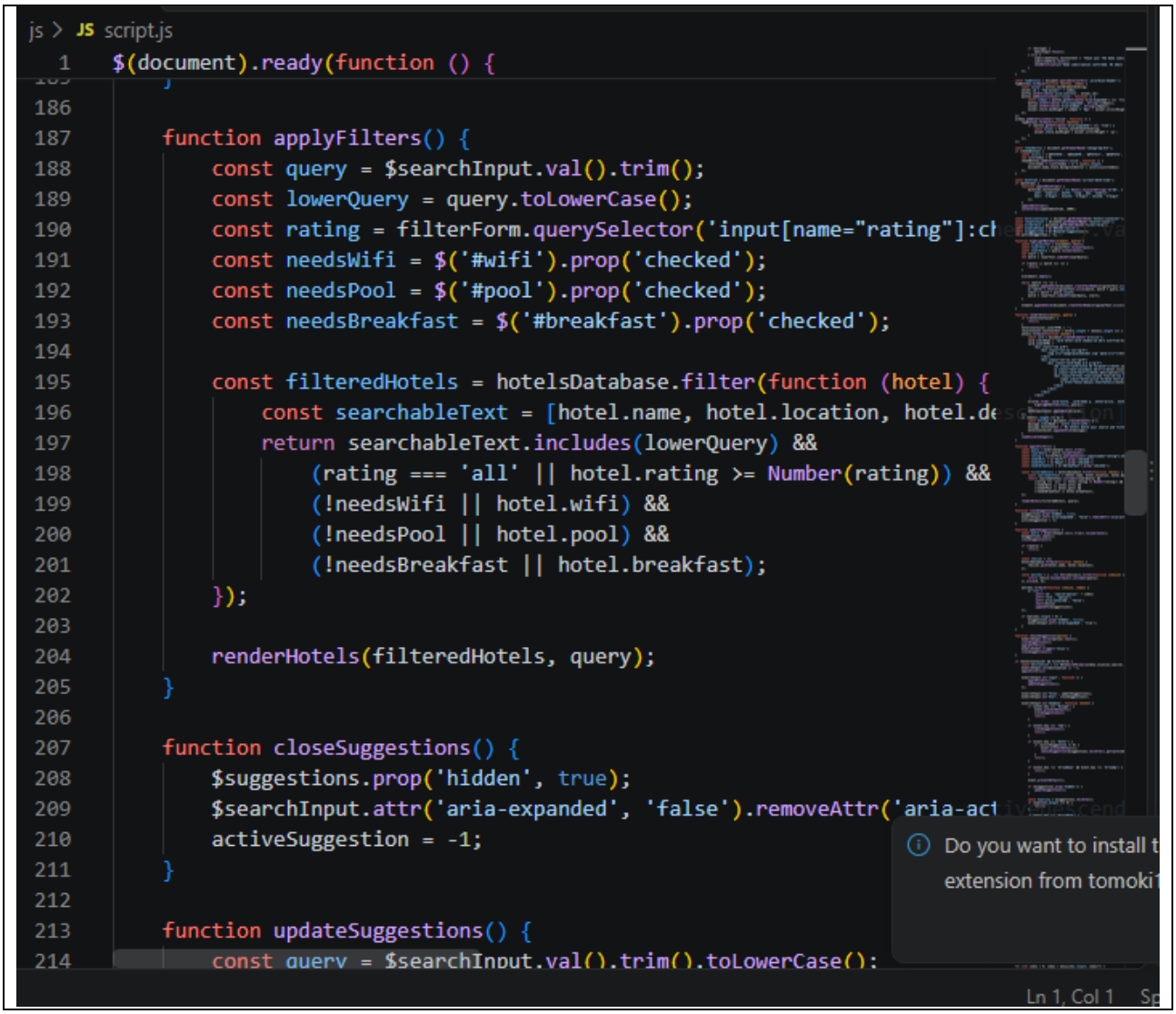
The page loads with jQuery 4.0.0.

Task 1 Live search

1. The Search page checks the input whenever the user types.
2. applyFilters() searches hotel names, cities and descriptions. Rating and amenity filters also apply.
3. The page updates the cards and result count without reloading.

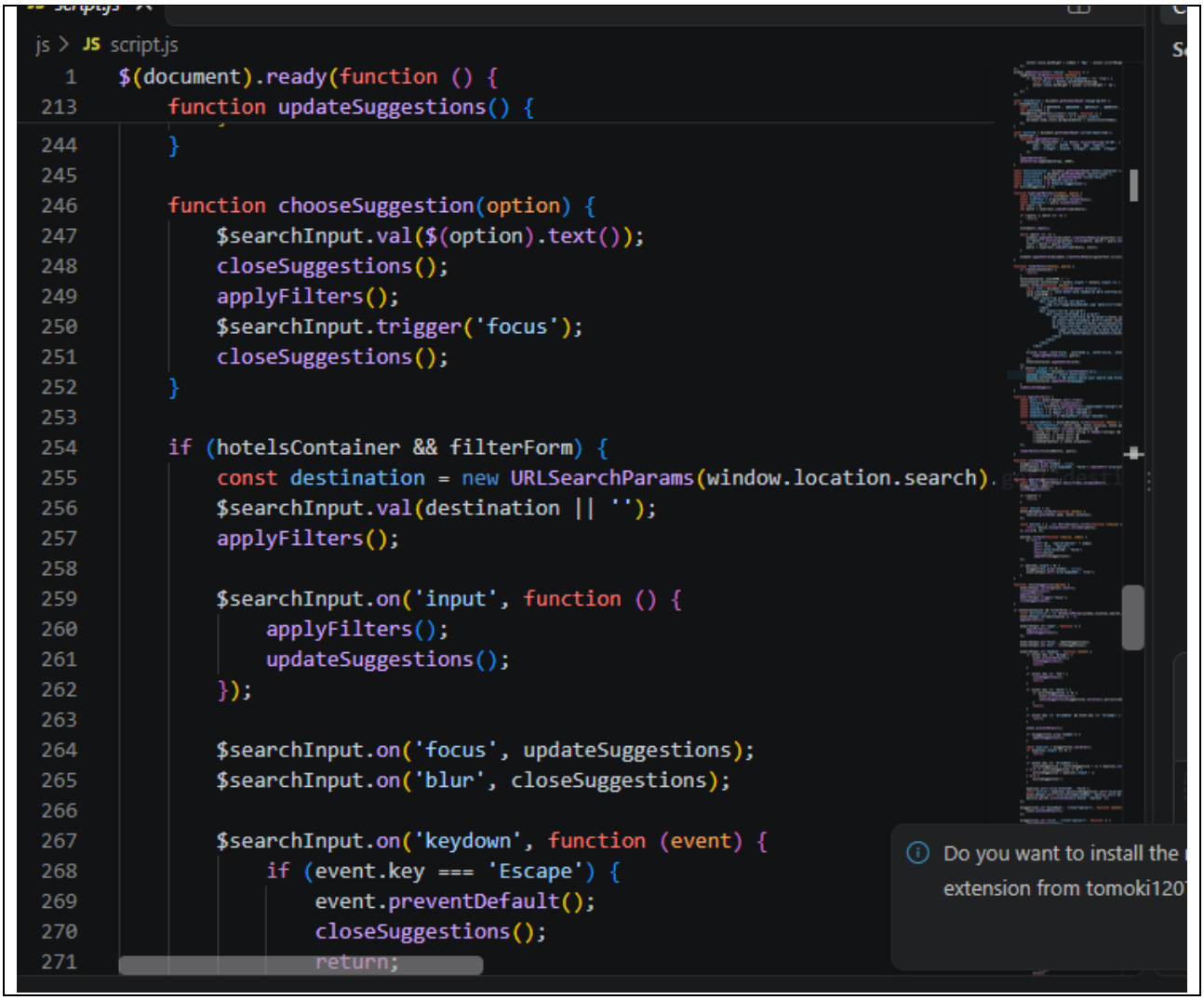
VS Code: js/script.js, lines 187-205

Show applyFilters() and the hotel filtering conditions.



VS Code: js/script.js,

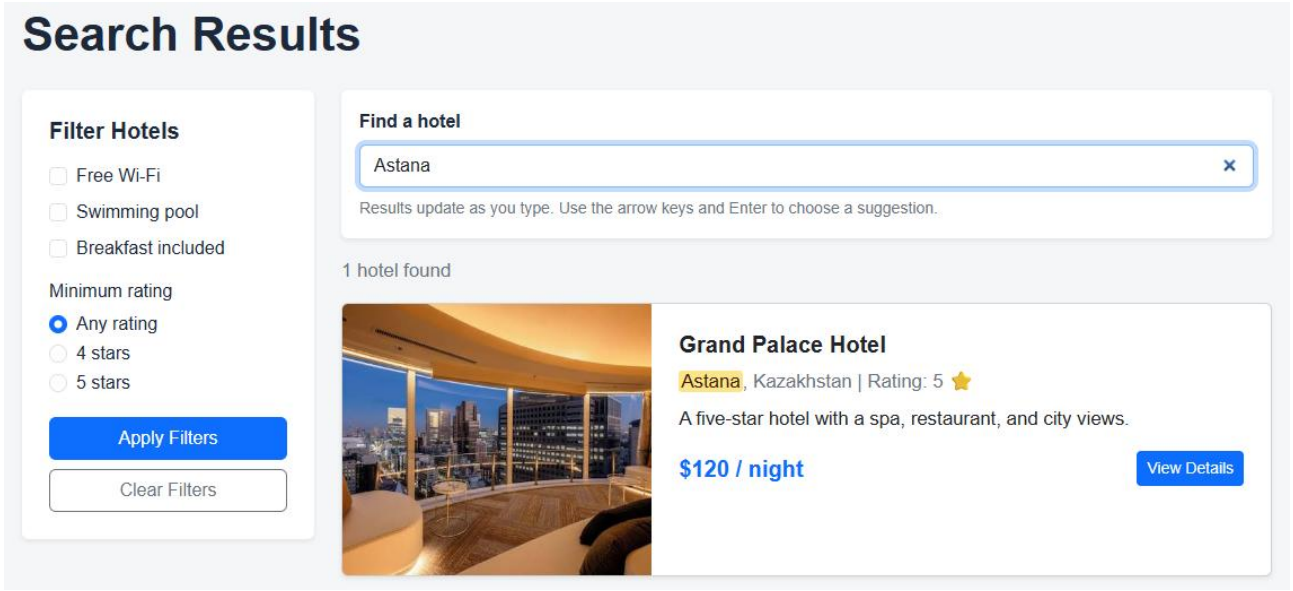
Show the input event that updates the results.



```
js > JS script.js
1   $(document).ready(function () {
213     function updateSuggestions() {
244     }
245
246     function chooseSuggestion(option) {
247         $searchInput.val($(option).text());
248         closeSuggestions();
249         applyFilters();
250         $searchInput.trigger('focus');
251         closeSuggestions();
252     }
253
254     if (hotelsContainer && filterForm) {
255         const destination = new URLSearchParams(window.location.search).
256         $searchInput.val(destination || '');
257         applyFilters();
258
259         $searchInput.on('input', function () {
260             applyFilters();
261             updateSuggestions();
262         });
263
264         $searchInput.on('focus', updateSuggestions);
265         $searchInput.on('blur', closeSuggestions);
266
267         $searchInput.on('keydown', function (event) {
268             if (event.key === 'Escape') {
269                 event.preventDefault();
270                 closeSuggestions();
271                 return;
```

Do you want to install the
extension from tomoki120

Website result



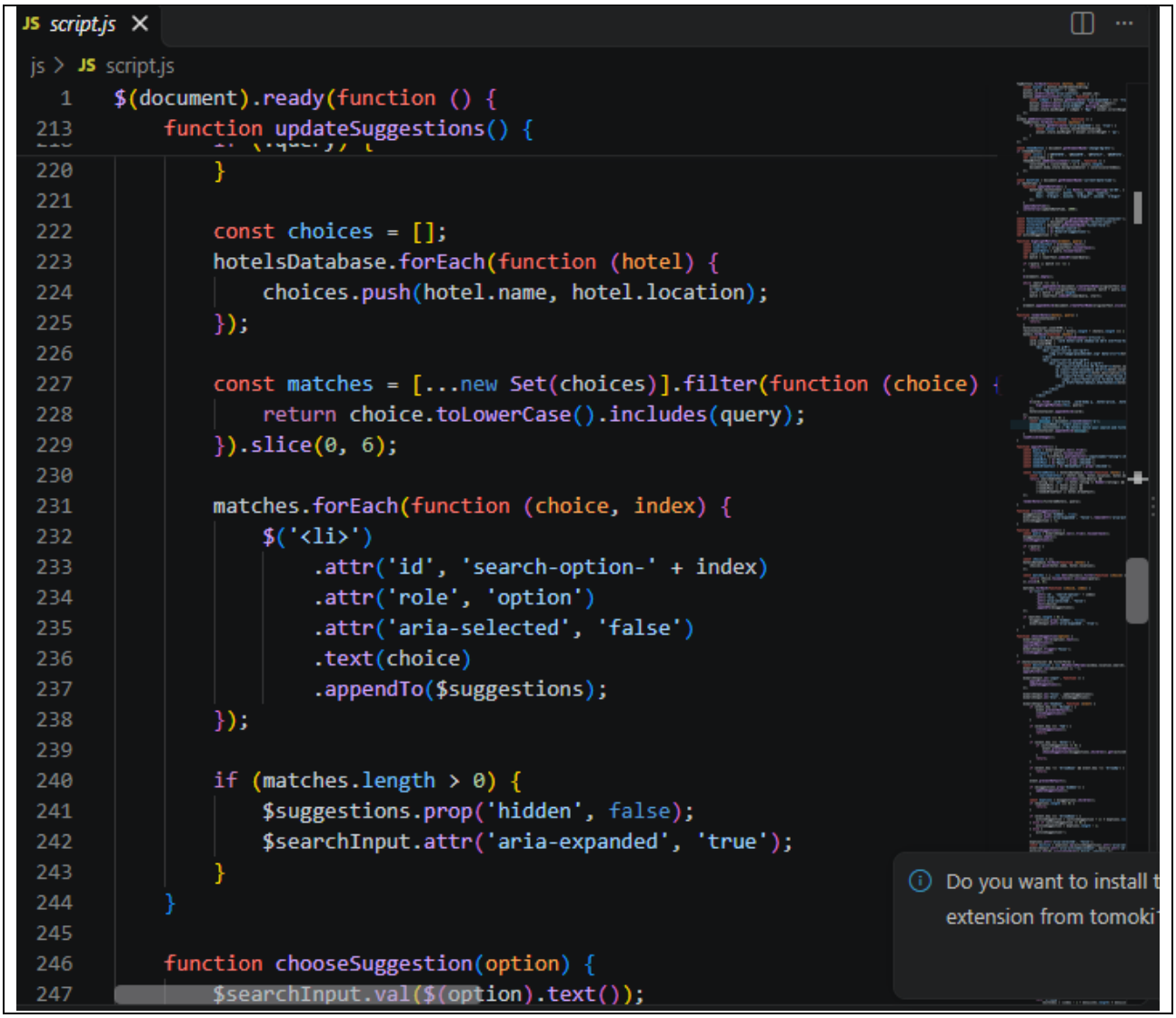
Typing Astana leaves one matching hotel.

Task 2 Autocomplete suggestions

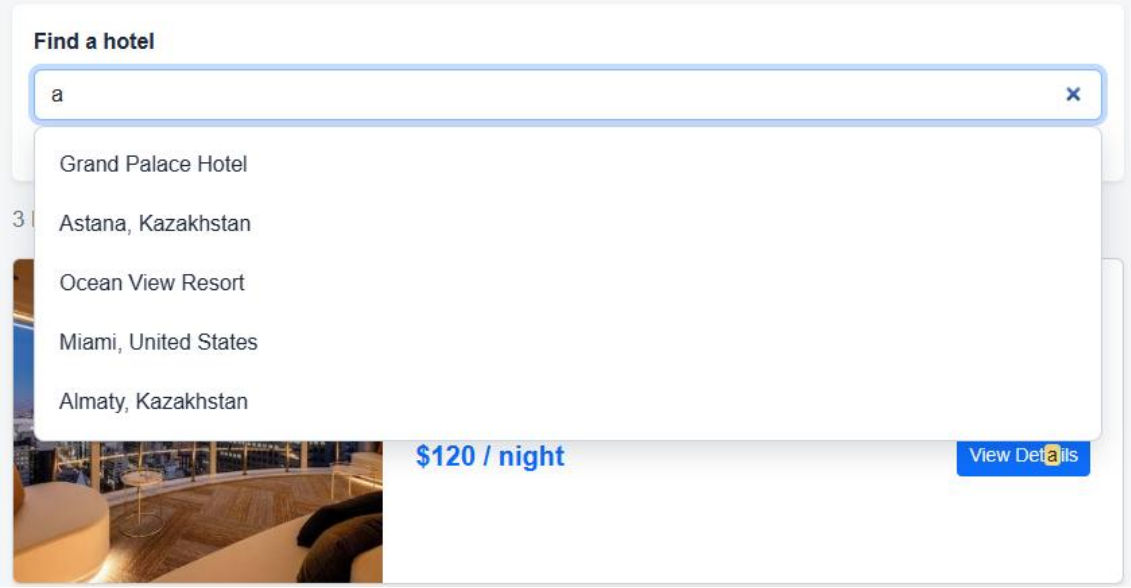
1. We collect hotel names and locations from hotelsDatabase.
2. updateSuggestions() shows up to six matching choices below the input.
3. The user can select a suggestion with a mouse click or the arrow keys and Enter.

VS Code: js/script.js, lines 222-244

Show how updateSuggestions() builds and displays the dropdown.



Website result



Suggestions appear as the user types.

Task 3 Search highlighting

1. highlightMatches() finds each occurrence of the search text without checking letter case.
2. Matching text is placed in a mark element. Other text stays unchanged.
3. Clearing the input removes the highlighting.

VS Code: js/script.js, lines 123-144

Show the complete highlightMatches() function.

```
$(document).ready(function () {
    const $searchInput = $('#hotel-search');
    const $suggestions = $('#search-suggestions');
    let activeSuggestion = -1;

    function highlightMatches(element, query) {
        const originalText = $(element).text();
        const lowerText = originalText.toLowerCase();
        const lowerQuery = query.toLowerCase();
        let start = 0;
        let match = lowerText.indexOf(lowerQuery);

        if (!query || match === -1) {
            return;
        }

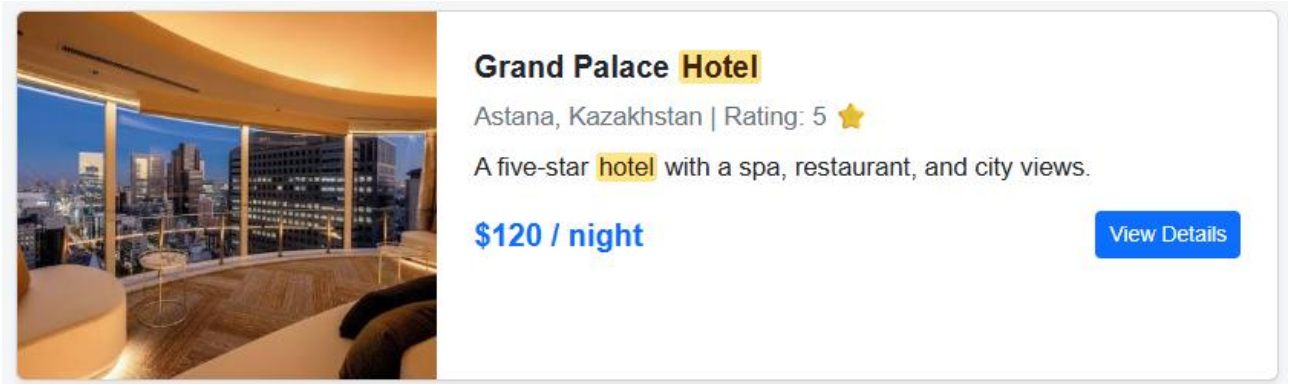
        $(element).empty();

        while (match !== -1) {
            element.appendChild(document.createTextNode(originalText.sli
            $('<mark>').text(originalText.slice(match, match + query.len
            start = match + query.length;
            match = lowerText.indexOf(lowerQuery, start);
        }

        element.appendChild(document.createTextNode(originalText
    }

    function renderHotels(hotels, query) {
        if (!hotelsContainer) {
```

Website result



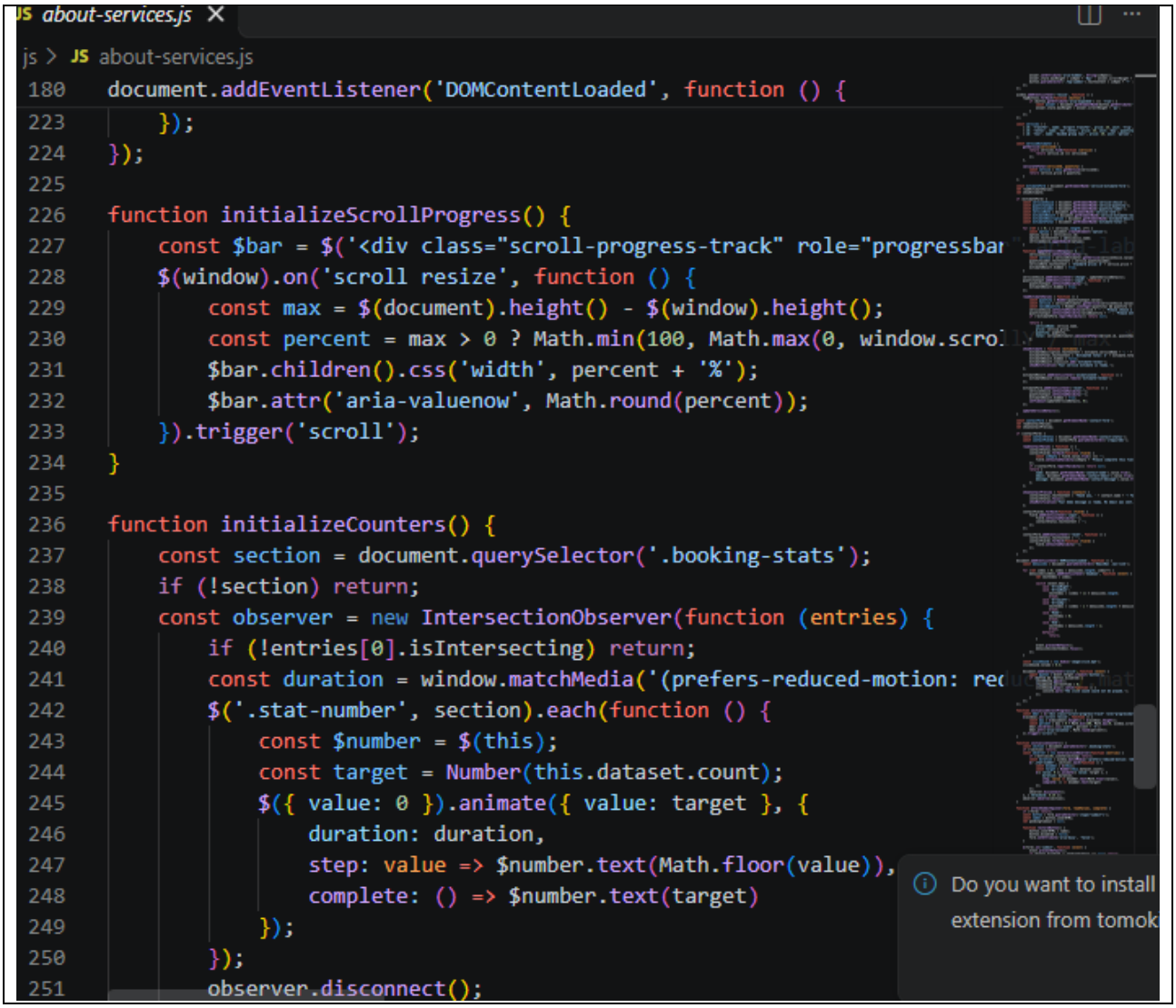
The matching words are highlighted in the hotel cards.

Task 4 Scroll progress bar

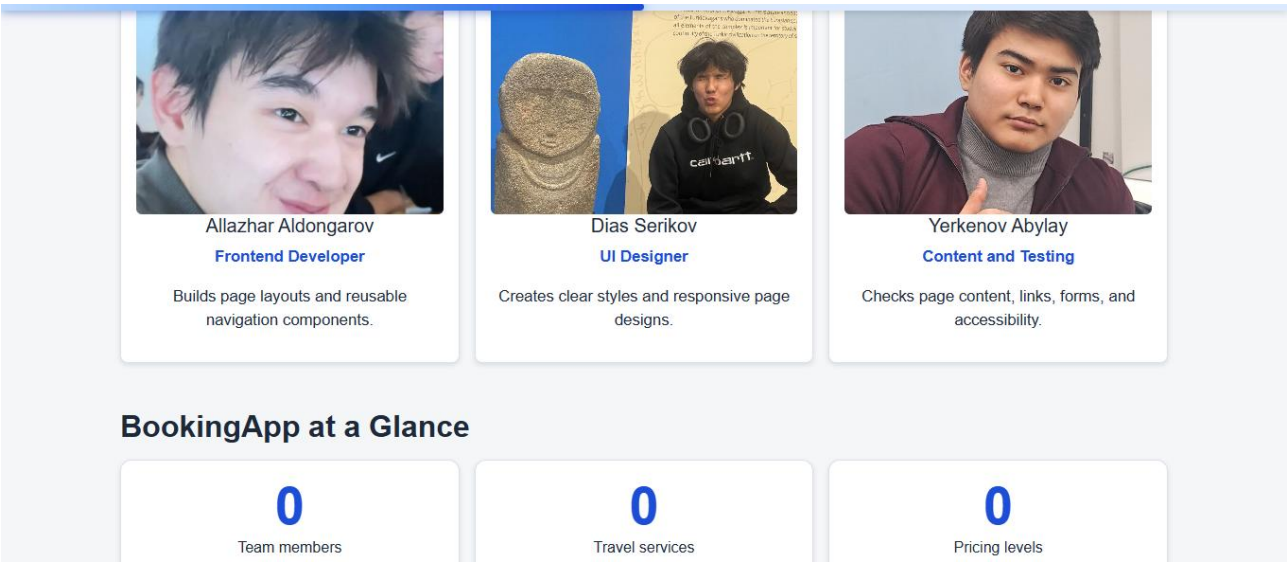
1. We added a fixed progress bar to About and Services.
2. The code divides the scroll position by the scrollable page height to find the percentage.
3. Scroll and resize events update the bar width from 0% to 100%.

VS Code: js/about-services.js, lines 226-234

Show initializeScrollProgress().



Website result



The bar at the top follows the scroll position.

Task 5 Animated number counters

1. The statistics on About start at zero. The target values are stored in data-count.
2. When the section enters the screen, jQuery animate() increases the numbers.
3. The counters stop at their targets and run only once.

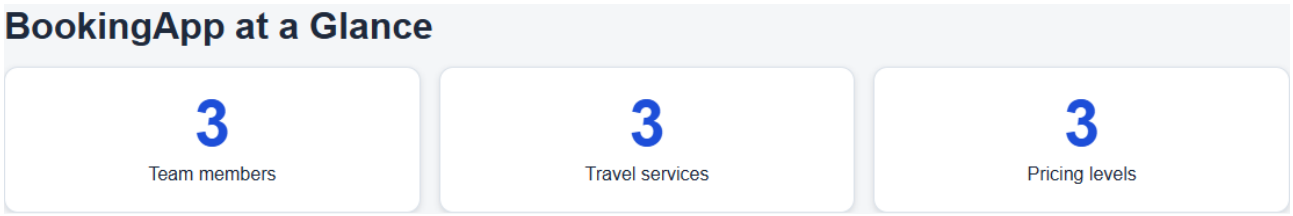
VS Code: js/about-services.js, lines 236-254

Show initializeCounters() and the number animation.

```
236  function initializeCounters() {
237      const section = document.querySelector('.booking-stats');
238      if (!section) return;
239      const observer = new IntersectionObserver(function (entries) {
240          if (!entries[0].isIntersecting) return;
241          const duration = window.matchMedia('(prefers-reduced-motion: rec
242          $('.stat-number', section).each(function () {
243              const $number = $(this);
244              const target = Number(this.dataset.count);
245              $({ value: 0 }).animate({ value: target }, {
246                  duration: duration,
247                  step: value => $number.text(Math.floor(value)),
248                  complete: () => $number.text(target)
249              });
250          });
251          observer.disconnect();
252      }, { threshold: 0.25 });
253      observer.observe(section);
254  }
255
256  function attachSubmitSpinner(form, readValues, complete) {
257      if (!form) return;
258      const button = form.querySelector('[type="submit"]');
259      const label = button.innerHTML;
260      let pendingTimeout = null;
```

Do you want to install the extension from tomoki12

Website result



The three counters finish at 3, 3 and 3.

Task 6 Loading spinner

1. The Contact form and service calculator first validate and save the submitted values.
2. The Submit button shows a spinner and "Please wait..." and becomes disabled.
3. After 1.2 seconds, the button returns to normal and the result appears. Clear cancels the waiting action.

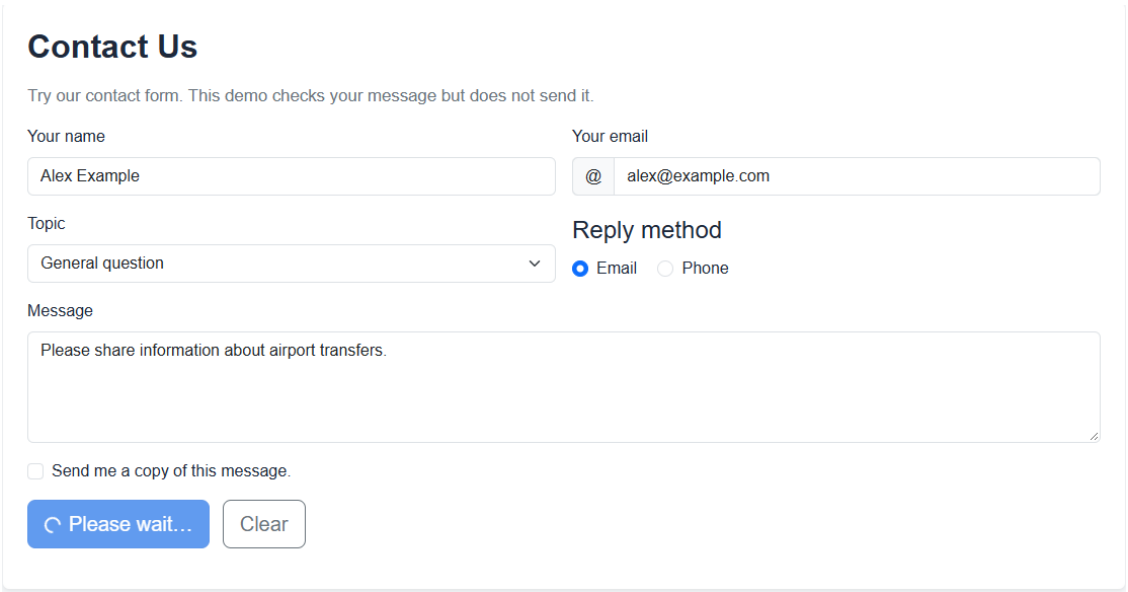
VS Code: js/about-services.js, lines 268-289

Show the submit handler, timeout and reset handler in attachSubmitSpinner().

```
268     $(form).on('submit', function (event) {
269         event.preventDefault();
270         if (button.disabled || pendingTimeout !== null) return;
271         const values = readValues();
272         if (!values) return;
273
274         button.innerHTML = '<span class="submit-spinner" aria-hidden="tr
275         button.disabled = true;
276         form.setAttribute('aria-busy', 'true');
277
278         pendingTimeout = setTimeout(function () {
279             pendingTimeout = null;
280             restoreButton();
281             complete(values);
282         }, 1200);
283     });
284
285     $(form).on('reset', function () {
286         clearTimeout(pendingTimeout);
287         pendingTimeout = null;
288         restoreButton();
289     });
290 }
```

Do you want to install th
extension from tomoki1

Website result



The Contact button shows the waiting state.

Task 7 Temporary notifications

1. showNotification() displays a message after a successful demo action.
2. jQuery fadeIn(), delay() and fadeOut() show the message and then hide it.
3. The same function is used for subscription, contact, service estimates and booking.

VS Code: js/common.js, lines 1-18

Show showNotification() and the fadeIn(), delay() and fadeOut() calls.

```
function showNotification(message, type = 'success') {
    let notification = $('#notification');

    if (notification.length === 0) {
        notification = $('<div>', {
            id: 'notification',
            class: 'app-notification',
            role: 'status',
            'aria-live': 'polite',
            'aria-atomic': 'true'
        }).appendTo('body');
    }

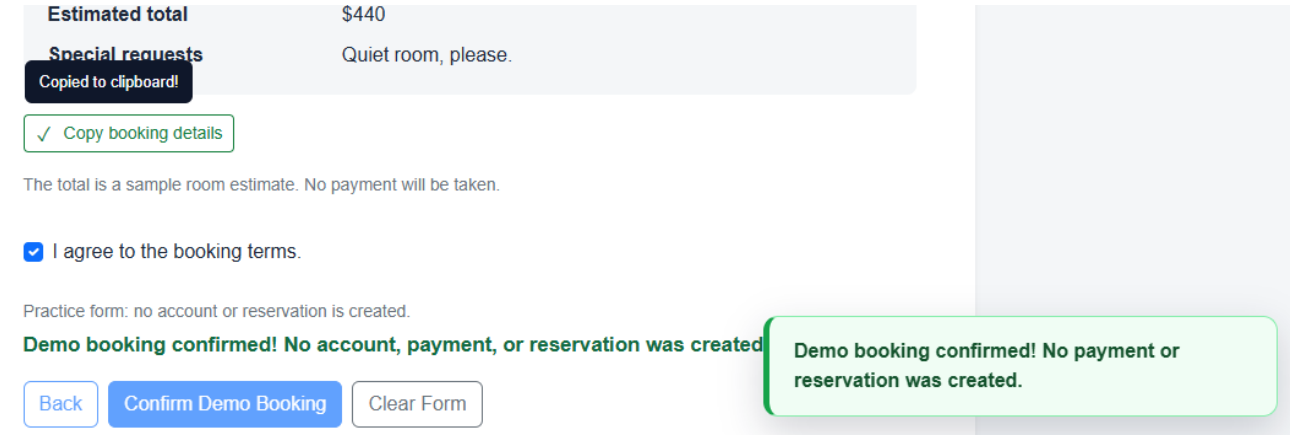
    notification.stop(true, true);
    notification.toggleClass('notification-error', type === 'error');
    notification.text(message);
    notification.fadeIn(180).delay(2600).fadeOut(350);
}

function loadVisibleImages() {
    const windowTop = $(window).scrollTop();
    const windowBottom = windowTop + $(window).height();

    $('img[data-src]').each(function () {
        const image = $(this);

        if (!image.is(':visible')) {
            return;
        }
```

Website result



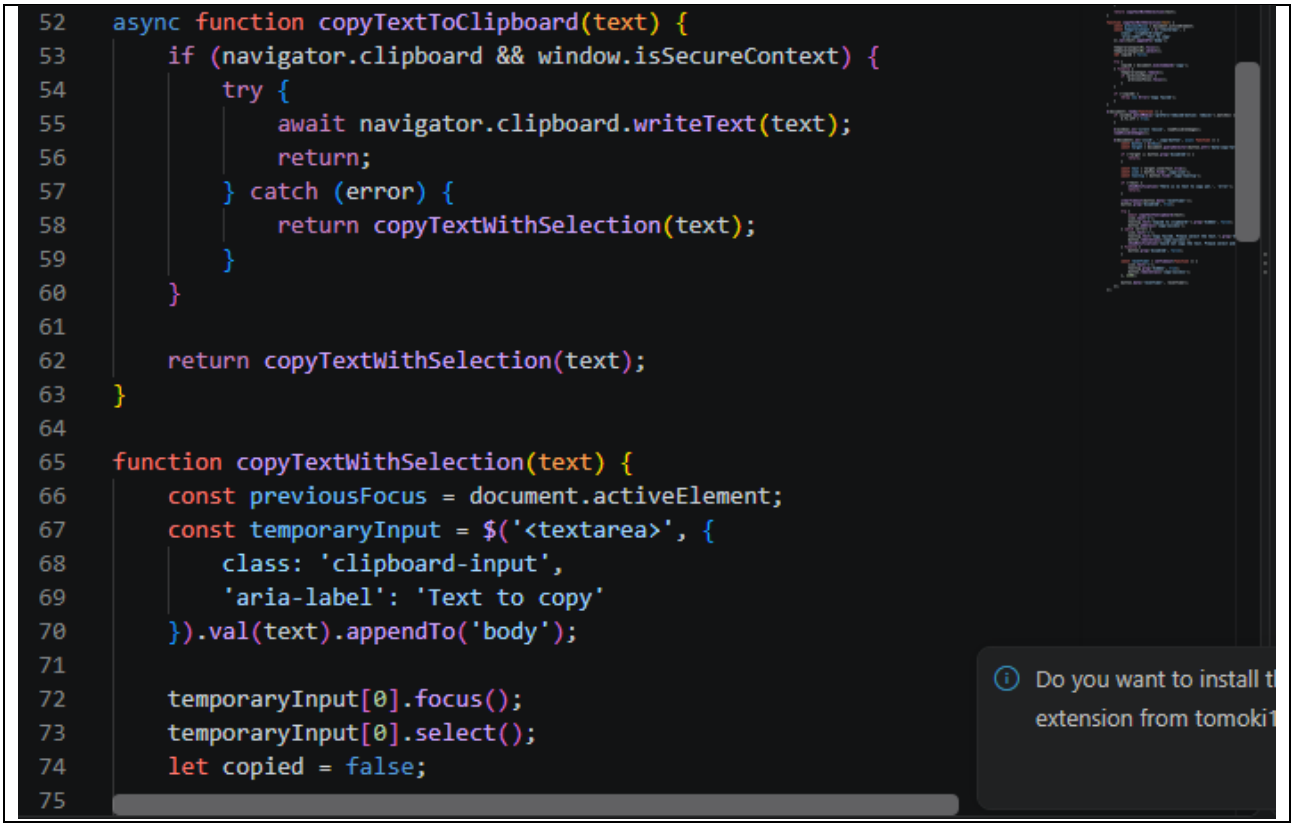
A successful action displays a temporary notification.

Task 8 Copy to clipboard

1. Copy buttons use data-copy-target to select the hotel location or booking summary.
2. copyTextToClipboard() writes the visible text to the clipboard.
3. After success, the button shows a checkmark and "Copied to clipboard!". It returns to normal after a short delay.

VS Code: js/common.js, lines 52-63

Show copyTextToClipboard().



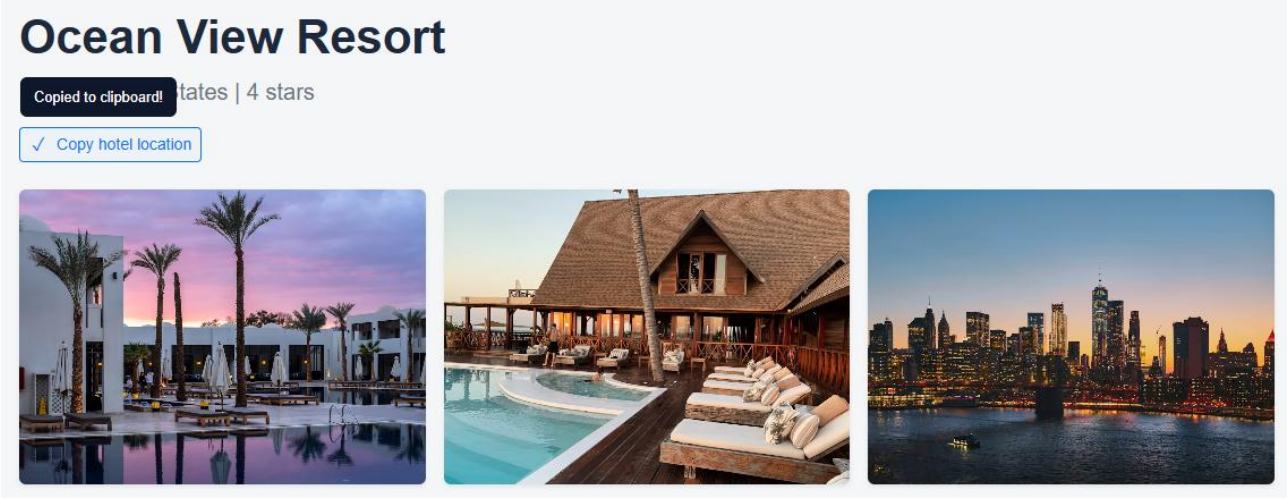
```
52  async function copyTextToClipboard(text) {
53      if (navigator.clipboard && window.isSecureContext) {
54          try {
55              await navigator.clipboard.writeText(text);
56              return;
57          } catch (error) {
58              return copyTextWithSelection(text);
59          }
60      }
61
62      return copyTextWithSelection(text);
63  }
64
65  function copyTextWithSelection(text) {
66      const previousFocus = document.activeElement;
67      const temporaryInput = $('<textarea>', {
68          class: 'clipboard-input',
69          'aria-label': 'Text to copy'
70      }).val(text).appendTo('body');
71
72      temporaryInput[0].focus();
73      temporaryInput[0].select();
74      let copied = false;
75
```

VS Code: js/common.js, lines 118-136

Show the success icon, tooltip and reset timer.

```
118         try {
119             await copyTextToClipboard(text);
120             icon.text('☑');
121             tooltip.text('Copied to clipboard!').prop('hidden', false);
122             button.addClass('copy-success');
123         } catch (error) {
124             icon.text('⧉');
125             tooltip.text('Copy failed. Please select the text.').prop('h
126             button.removeClass('copy-success');
127             showNotification('Could not copy the text. Please select and
128         } finally {
129             button.prop('disabled', false);
130         }
131
132         const resetTimer = setTimeout(function () {
133             icon.text('⧉');
134             tooltip.prop('hidden', true);
135             button.removeClass('copy-success');
136         }, 2200);
137
138         button.data('resetTimer', resetTimer);
139     });
```

Website result



The Copy button confirms that the text was copied.

Task 9 Image lazy loading

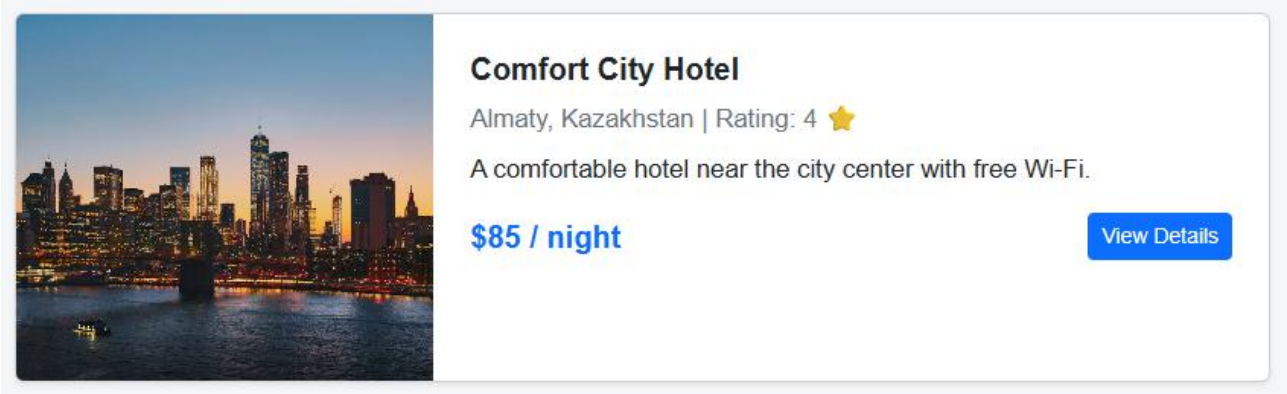
1. Hotel cards on Search start with a placeholder. The real image path is stored in data-src.
2. loadVisibleImages() checks which images are inside the screen.
3. For a visible image, the code sets src and removes data-src. The check runs after scrolling and after new cards are shown.

VS Code: js/common.js, lines 24-48

Show the visibility check and the change from data-src to src.

```
24      $('img[data-src]').each(function () {
25          const image = $(this);
26
27          if (!image.is(':visible')) {
28              return;
29          }
30
31          const imageTop = image.offset().top;
32          const imageBottom = imageTop + image.outerHeight();
33
34          if (imageTop < windowBottom && imageBottom > windowTop) {
35              const imagePath = image.attr('data-src');
36
37              image.one('load', function () {
38                  image.addClass('image-loaded');
39              });
40
41              image.one('error', function () {
42                  image.attr('src', 'image/placeholder.svg');
43                  image.attr('alt', image.attr('alt') + ' - image unavail
44              });
45
46              image.removeAttr('data-src');
47              image.attr('src', imagePath);
48          }
49      });
50  }
51
```

Website result



The lower hotel image loads after scrolling down.

Conclusion

The main challenge was keeping the new search and existing filters working together. We solved it with one filtering function. We also saved form values before the loading delay so that editing or clearing a form does not produce a wrong result. All tasks were tested, and the updated website is deployed on GitHub Pages.